Avance de Proyecto de Aula

Sistema de Detección de Fatiga y Somnolencia en Conducción

Visión por computador aplicada a la seguridad vial

Asignatura: Ingeniería de Software

Programa: Ingeniería de Sistemas y Computación

Institución: Universidad Tecnológica de Bolívar

Equipo: Daniel Sanchez Araujo - Sebastián David Ruiz Ortega - Juan Diego Serpa Medina

Fecha: Septiembre 17 de 2026

1. Descripción del problema

1.1. Contexto

La fatiga y la somnolencia al volante constituyen uno de los factores de riesgo más subestimados dentro de la siniestralidad vial. A diferencia del consumo de alcohol o del exceso de velocidad, la fatiga no deja evidencia física verificable después del siniestro, por lo que tiende a subregistrarse en los informes de accidentalidad. Sin embargo, la literatura especializada en seguridad vial coincide en que los episodios de microsueño (lapsos de atención de entre uno y varios segundos) son responsables de una proporción significativa de los accidentes en carretera, particularmente en trayectos largos, en horarios nocturnos y en la conducción profesional de carga y pasajeros.

1.2. Problema identificado

No existe, en el contexto de las pequeñas y medianas empresas de transporte del país, un mecanismo accesible que permita **detectar en tiempo real** los signos objetivos de fatiga en el conductor y **alertar de inmediato** antes de que el episodio derive en un siniestro. Las soluciones comerciales existentes (sistemas DMS integrados de fábrica o dispositivos de posventa de marcas internacionales) presentan tres barreras concretas:

- **Costo**: requieren inversión por vehículo que resulta prohibitiva para flotas pequeñas.
- **Dependencia del proveedor**: operan como cajas negras, sin posibilidad de ajustar umbrales de sensibilidad ni de integrar los datos con sistemas propios.
- **Ausencia de trazabilidad para el supervisor**: muchas soluciones alertan al conductor pero no consolidan un histórico consultable que permita a la empresa identificar patrones de riesgo por conductor o por ruta.

1.3. Oportunidad

La disponibilidad actual de bibliotecas de visión por computador de código abierto — particularmente MediaPipe Face Mesh, que entrega 468 puntos de referencia faciales sin requerir GPU dedicada — permite construir un sistema de detección multiseñal que corre sobre hardware de bajo costo (una laptop estándar o un mini-PC tipo Raspberry Pi con cámara USB). Esto abre la posibilidad de una solución accesible, auditable y adaptable al contexto local.

1.4. Usuarios y beneficiarios

Tipo	Descripción
Usuario directo	**Conductor**: recibe la alerta local sonora y visual en el momento en que se detecta el episodio de fatiga.
Usuario directo	**Supervisor de flota**: consulta el dashboard web, filtra incidentes y toma decisiones operativas sobre asignación de rutas y descansos.
Usuario directo	**Administrador del sistema**: gestiona el registro de conductores y vehículos, y configura los parámetros de sensibilidad.
Beneficiario indirecto	**Empresa de transporte**: reduce exposición a siniestros, costos asociados y responsabilidad legal.
Beneficiario indirecto	**Sociedad y otros actores viales**: se reduce la probabilidad de siniestros que involucran a terceros.

1.5. Objetivo general

Desarrollar un sistema de detección de fatiga y somnolencia en conducción, basado en visión por computador, que identifique en tiempo real señales objetivas de fatiga — cierre prolongado de ojos, bostezos y cabeceo —, emita alertas locales inmediatas al conductor y registre cada incidente en una plataforma web que permita a los supervisores consultar, filtrar y analizar el histórico de eventos.

2. Identificación de interesados

Se aplicó un análisis de interesados a partir de la revisión documental del plan de desarrollo y de la sesión de lluvia de ideas del equipo,. Se identificaron siete interesados, clasificados según su relación con el sistema.

Interesado	Relación	Descripción del rol	Necesidades y expectativas
Conductor	Usuario final (directo)	Persona que opera el vehículo y frente a quien se ubica la cámara. Es el sujeto monitoreado y el receptor de la alerta local.	Recibir una alerta perceptible e inmediata sin que resulte invasiva. Que el sistema no genere alertas falsas constantes que terminen por ignorarse. Claridad sobre qué datos se registran sobre su desempeño.
Supervisor de flota	Usuario final (directo)	Responsable operativo de asignar rutas y turnos. Consume la información consolidada del dashboard.	Consultar rápidamente qué conductores presentan más incidentes, filtrar por fecha y tipo de evento, e interpretar la información sin formación técnica previa.
Administrador del sistema	Usuario final (directo)	Perfil con privilegios para gestionar los catálogos de conductores y vehículos y configurar parámetros del sistema.	Realizar operaciones CRUD sobre conductores y vehículos. Ajustar umbrales de sensibilidad según el tipo de operación. Administrar las credenciales de acceso al dashboard.
Empresa de transporte	Cliente / patrocinador	Organización que adquiere e implanta la solución en su flota.	Reducir siniestralidad y costos asociados. Disponer de evidencia documentada para procesos de auditoría y para aseguradoras. Costo de implantación proporcional al tamaño de la flota.
Equipo de desarrollo	Ejecutor técnico	Tres desarrolladores con responsabilidades diferenciadas: visión por computador, *backend* y *frontend*.	Contratos de interfaz definidos tempranamente para trabajar en paralelo. Alcance acotado y estable. Entorno reproducible mediante contenedores.
Docente / tutor	Evaluador académico	Valida el cumplimiento de los objetivos de aprendizaje y la calidad del proceso de ingeniería.	Trazabilidad entre elicitación, requerimientos y modelado. Evidencia real del trabajo del equipo en el repositorio y en la herramienta de gestión.
Entidad reguladora	Interesado externo	Autoridades de tránsito y marco normativo de protección de datos personales.	Cumplimiento de la Ley 1581 de 2012 en el tratamiento de datos biométricos y personales del conductor. Consentimiento informado para el monitoreo.

3. Técnicas de elicitación aplicadas

Se aplicaron cuatro técnicas de elicitación complementarias. La selección respondió a dos criterios: (i) la posibilidad real de ejecutarlas con un equipo de tres personas y en el tiempo disponible, y (ii) su capacidad para cubrir dimensiones distintas del problema — la dimensión técnica, la dimensión del usuario y la dimensión del mercado.

3.1. Técnica 1: Revisión de documentos existentes

Objetivo	Establecer la base conceptual del problema y determinar qué métricas de fatiga son medibles mediante visión por computador con hardware accesible.
Participantes	Los tres integrantes del equipo (revisión distribuida por temas).
Periodo	Semana 1 y 2 del proyecto.
Herramientas	Buscadores académicos, documentación oficial de MediaPipe y OpenCV, normativa colombiana en línea, gestor de referencias compartido.

Fuentes revisadas

- Literatura sobre el indicador *Eye Aspect Ratio* y su uso en detección de parpadeo y microsueño.
- Documentación técnica de MediaPipe Face Mesh (modelo de 468 puntos de referencia).
- Ley 1581 de 2012 sobre protección de datos personales en Colombia.
- Documentación de FastAPI y de esquemas de autenticación con JWT.

Principales hallazgos

- El EAR por sí solo no distingue un parpadeo normal de un microsueño: la variable discriminante es la *duración* del cierre, no su ocurrencia. Esto obligó a formular el requerimiento en términos de umbral temporal configurable (RF-03).
- La detección monoseñal produce tasas altas de error. La combinación de EAR, MAR y orientación de la cabeza en un puntaje agregado es el enfoque predominante, lo que sustenta directamente el requerimiento RF-06.
- El tratamiento de imágenes faciales constituye tratamiento de datos personales bajo la Ley 1581 de 2012, lo que genera el requerimiento no funcional RNF-03 y la decisión de no almacenar video sino únicamente metadatos del evento.
- MediaPipe opera en tiempo real sobre CPU, lo que hace viable la restricción de hardware de bajo costo (RNF-05).

3.2. Técnica 2: Benchmarking de soluciones similares

Objetivo	Identificar qué funcionalidades ofrece el mercado, qué brechas quedan sin cubrir para flotas pequeñas y qué características debe adoptar el MVP para ser competitivo en el contexto local.
Participantes	Los tres integrantes del equipo.
Periodo	Semana 2 del proyecto.
Herramientas	Sitios web oficiales de los fabricantes, fichas técnicas públicas, matriz comparativa en hoja de cálculo compartida.
Principales hallazgos	
▪ Los sistemas integrados de fábrica no son aplicables a flotas con vehículos de modelos heterogéneos, lo que confirma la decisión de diseñar un cliente por software independiente del vehículo.	

3.3. Técnica 3: Lluvia de ideas del equipo

Objetivo	Generar el conjunto inicial de funcionalidades candidatas y delimitar qué entra en el alcance del MVP frente a lo que se pospone.
Participantes	Los tres integrantes del equipo.
Periodo	Semana 2 del proyecto, sesión de aproximadamente 90 minutos.
Herramientas	Tablero digital colaborativo con notas agrupadas por categoría; consolidación posterior en el *backlog* priorizado.
Principales hallazgos	
▪ Se generaron 27 ideas, de las cuales 14 se conservaron dentro del alcance del MVP.	
▪ Surgió el escenario de pérdida de conectividad en carretera. El equipo concluyó que la alerta local debe ser independiente del *backend*, lo que produjo el requerimiento no funcional RNF-04 y determinó el desacoplamiento de los tres componentes de la arquitectura.	
▪ Se descartaron explícitamente: aplicación móvil nativa, monitoreo simultáneo de múltiples cámaras por vehículo, reconocimiento de identidad del conductor por rostro y modelo de aprendizaje adaptativo por conductor.	

4. Muestras de las herramientas utilizadas

4.1. Tablero de lluvia de ideas

Resultado consolidado de la sesión de la semana 2. Las ideas se agruparon por componente y se marcaron según la decisión del equipo.

Categoría	**Idea generada**	**Decisión**
Detección	Detectar cierre prolongado de ojos	Incluida (RF-03)
Detección	Detectar bostezos	Incluida (RF-04)
Detección	Detectar cabeceo mediante orientación de la cabeza	Incluida (RF-05)
Detección	Combinar señales en un nivel de riesgo	Incluida (RF-06)
Detección	Detectar uso del teléfono móvil	Descartada (fuera de alcance)
Detección	Reconocer la identidad del conductor por rostro	Descartada (riesgo normativo)
Detección	Modelo que aprenda el patrón de cada conductor	Descartada (MVP+)
Alerta	Alerta sonora local	Incluida (RF-07)
Alerta	Alerta visual en pantalla	Incluida (RF-07)
Alerta	Escalamiento de alerta según severidad	Incluida parcialmente (RF-06)
Alerta	Notificación en tiempo real al supervisor	Pospuesta (WebSocket opcional)
Alerta	Vibración en el asiento	Descartada (requiere hardware adicional)
Registro	Registrar cada evento con tipo, severidad y hora	Incluida (RF-08)
Registro	Enviar eventos al servidor vía API REST	Incluida (RF-09)
Registro	Funcionar sin conexión y sincronizar después	Incluida (RNF-04)
Registro	Guardar clip de video del episodio	Descartada (normativa y almacenamiento)
Dashboard	Mapa con ubicación GPS del incidente	Descartada (requiere módulo GPS)
Dashboard	Aplicación móvil para el supervisor	Descartada (MVP+)
Gestión	CRUD de conductores y vehículos	Incluida (RF-11)
Gestión	Módulo de turnos y planificación de rutas	Descartada (fuera de alcance)
Gestión	Integración con sistemas de nómina	Descartada (fuera de alcance)

Resumen: 27 ideas generadas, 14 incorporadas al alcance del MVP, 4 pospuestas como MVP+ y 9 descartadas explícitamente.

4.2. Otras evidencias

- **Repositorio de código:** el historial de confirmaciones refleja la evolución del trabajo por integrante (ver sección 8 del documento).

5. Requerimientos identificados

5.1. Requerimientos funcionales

ID	Nombre	Descripción	Fuente	Prior.	Criterio de aceptación
RF-01	Captura de video en tiempo real	El sistema debe capturar video de forma continua desde una cámara web o embebida orientada al conductor.	Lluvia de ideas	Alta	El cliente inicia la captura y muestra el flujo de video sin interrupciones durante al menos 30 minutos continuos.
RF-02	Detección facial y extracción de landmarks	El sistema debe detectar el rostro del conductor y extraer los puntos de referencia faciales mediante MediaPipe Face Mesh.	Revisión documental	Alta	Con un rostro presente en el encuadre, el sistema entrega los 468 puntos de referencia en al menos el 95 % de los fotogramas procesados.
RF-03	Cálculo de EAR y cierre prolongado de ojos	El sistema debe calcular el *Eye Aspect Ratio* y detectar el cierre sostenido de ojos según un umbral de duración configurable.	Revisión documental	Alta	Ante un cierre de ojos que supera el umbral configurado, el sistema registra el evento; un parpadeo de duración normal no lo activa.
RF-04	Cálculo de MAR y detección de bostezo	El sistema debe calcular el *Mouth Aspect Ratio* para identificar bostezos.	Revisión documental	Alta	Ante un bostezo simulado, el sistema lo clasifica como tal en al menos 8 de 10 repeticiones controladas.
RF-05	Estimación de la orientación de la cabeza	El sistema debe estimar los ángulos de inclinación y giro de la cabeza para detectar cabeceo.	Revisión documental	Alta	El sistema reporta los ángulos y genera un evento de cabeceo cuando la inclinación supera el umbral definido de forma sostenida.
RF-06	Cálculo del nivel de riesgo de fatiga	El sistema debe combinar las señales de ojos, boca y cabeza en un puntaje o nivel de riesgo agregado.	Revisión documental	Alta	Ante la concurrencia de dos o más señales, el nivel de riesgo reportado es superior al obtenido con una sola señal.
RF-07	Emisión de alerta local	El sistema debe emitir una alerta sonora y visual en el puesto del conductor al superarse los umbrales definidos.	Lluvia de ideas	Alta	La alerta se emite en menos de 1 segundo desde la detección y es audible en condiciones normales de cabina.
RF-08	Registro de eventos	El sistema debe registrar cada evento detectado con tipo, severidad, fecha, hora y conductor asociado.	Lluvia de ideas	Alta	Cada evento detectado genera un registro con los cinco campos completos y consultable posteriormente.
RF-09	Envío de eventos al backend	El cliente debe transmitir los eventos registrados al servicio de backend mediante una API REST.	Lluvia de ideas	Alta	Un evento generado en el cliente aparece persistido en la base de datos en menos de 5 segundos con conectividad disponible.
RF-10	Gestión de conductores y vehículos	El sistema debe permitir crear, consultar, actualizar y eliminar registros de conductores y de vehículos.	Lluvia de ideas	Media	Las cuatro operaciones se ejecutan correctamente sobre ambas entidades y los cambios se reflejan en las consultas.
RF-11	Indicadores agregados	El dashboard debe mostrar indicadores consolidados, como incidentes por conductor y por periodo.	Entrevista	Alta	La vista muestra al menos dos gráficas agregadas cuyos totales coinciden con los registros de la tabla de incidentes.

5.2. Requerimientos no funcionales

ID	Nombre	Descripción	Fuente	Prior.	Criterio de aceptación
RNF-01	Rendimiento del procesamiento	El módulo de detección debe procesar el video a un mínimo de 15 FPS con latencia inferior a 200 ms por fotograma, sobre CPU y sin GPU dedicada.	Revisión documental	Alta	En pruebas sobre hardware estándar, la tasa media de fotogramas es $\geq$ 15 FPS y la latencia media por fotograma es $<$ 200 ms.
RNF-02	Disponibilidad sin conectividad	El cliente de detección debe seguir emitiendo alertas locales aunque se pierda la conexión con el backend, y sincronizar los eventos al restablecerse.	Lluvia de ideas	Alta	Con la red desconectada, las alertas siguen emitiéndose; al restablecer la conexión, los eventos pendientes se persisten sin pérdida ni duplicación.
RNF-03	Seguridad y protección de datos	El tratamiento de los datos de los conductores debe cumplir la Ley 1581 de 2012; el acceso al dashboard requiere autenticación y no se almacena video.	Revisión documental	Alta	No existen archivos de video en el almacenamiento; todo endpoint con datos exige token válido; se documenta el consentimiento informado.
RNF-04	Escalabilidad	La arquitectura debe permitir incorporar vehículos o cámaras adicionales sin rediseño mayor del backend.	Benchmarking	Media	Se agrega un segundo cliente de detección y el backend registra sus eventos correctamente sin cambios en el código.
RNF-05	Portabilidad de hardware	El cliente de detección debe ejecutarse en hardware de bajo costo, como una laptop estándar o un mini-PC con cámara USB.	Revisión documental	Alta	El cliente se ejecuta y cumple RNF-01 en al menos dos configuraciones de hardware distintas sin GPU dedicada.
RNF-06	Mantenibilidad	El código debe estar modularizado en captura, procesamiento, alertas y comunicación, y debe estar documentado.	Lluvia de ideas	Media	Cada módulo reside en su propio paquete con interfaz definida; existe documentación técnica de cada uno.
RNF-07	Compatibilidad de navegadores	El dashboard debe funcionar correctamente en los navegadores de escritorio de mayor uso.	Benchmarking	Media	Todas las vistas se renderizan y operan sin errores en Chrome, Edge y Firefox en sus versiones estables.
RNF-08	Portabilidad de despliegue	El backend y la base de datos deben poder desplegarse mediante contenedores en cualquier entorno, local o en nube.	Lluvia de ideas	Media	Con un único comando de composición, los servicios quedan operativos en una máquina limpia.

6. Priorización de requerimientos

6.1. Criterio de priorización

Se aplicó el método **MoSCoW** (*Must have*, *Should have*, *Could have*, *Won't have*), ya adoptado en el plan de desarrollo del proyecto para delimitar el alcance del MVP. La asignación de cada requerimiento a una categoría se hizo evaluando tres factores:

1. **Criticidad funcional.** Si el requerimiento se elimina, ¿sigue existiendo el producto? Un sistema que no detecta fatiga o que no alerta deja de ser el producto propuesto; uno que no exporta a CSV sigue siéndolo.
2. **Dependencia técnica.** Los requerimientos de los que dependen otros se priorizan antes. La detección facial (RF-02) condiciona a RF-03, RF-04 y RF-05, por lo que necesariamente es *Must*.
3. **Impacto en el interesado principal.** Los hallazgos de la entrevista elevaron la prioridad de los requerimientos vinculados a la utilidad real para el supervisor, particularmente RF-11, que inicialmente se había considerado secundario.

6.2. Clasificación MoSCoW

Categoría	Requerimientos	Justificación
Must have (Obligatorio)	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-10 RNF-01, RNF-02, RNF-03, RNF-05	Constituyen el núcleo indispensable: la cadena completa de captura, detección multiseñal, alerta, registro, transmisión y consulta. Sin cualquiera de ellos el sistema no cumple su objetivo general ni permite validar las hipótesis del proyecto.
Should have (Importante)	RF-10 RNF-04, RNF-06, RNF-07, RNF-8	Aportan valor operativo y calidad relevante, pero el sistema puede demostrarse funcionalmente sin ellos en una primera iteración: los datos de conductores pueden cargarse inicialmente de forma directa en la base de datos.
Could have (Deseable)	RF-10	Identificados como diferenciadores frente al mercado y solicitados por el supervisor, pero no bloquean la operación del MVP. Se abordan en el sprint 6 si el margen de tiempo lo permite.
Won't have (No incluido por ahora)	Aplicación móvil nativa Monitoreo multicámara simultáneo Identificación biométrica del conductor Geolocalización del incidente Modelo adaptativo por conductor Almacenamiento de video del episodio	Quedan explícitamente fuera del alcance. Cada uno implica dependencias de hardware, esfuerzo de desarrollo o riesgo normativo que exceden la capacidad del equipo de tres integrantes en 12 semanas. Se documentan para una eventual evolución posterior del producto.

6.3. Alcance de la primera versión funcional

La primera versión funcional del sistema incorporará la totalidad de los requerimientos clasificados como **Must have**, lo que corresponde a 13 requerimientos funcionales y 6 no funcionales. Esta versión permite demostrar el ciclo completo del producto: el cliente captura video, detecta las tres señales de fatiga, calcula el nivel de riesgo, alerta al conductor, registra el evento, lo transmite al backend y lo hace consultable por un supervisor autenticado a través del dashboard con filtros e indicadores agregados.

7. Validación y ajuste del modelado previo

7.1. Concepción inicial del modelado

El modelado UML entregado en el avance anterior ya definía los tres actores directos del sistema (Conductor, Supervisor y Administrador), la arquitectura de tres capas —cliente de captura embebido en Raspberry Pi, backend API REST con PostgreSQL sobre un servidor *on-premise*, y dashboard web como SPA— y el flujo principal de detección, alerta y registro de eventos de fatiga.

7.2. Hallazgos de la elicitación frente al modelado

El proceso de ingeniería de requerimientos (secciones 3 a 6) confirmó la mayor parte del diseño inicial, pero expuso un vacío relevante: **ningún diagrama representaba qué ocurre cuando el cliente de captura pierde conectividad con el backend**. La técnica de lluvia de ideas identificó explícitamente el escenario de pérdida de conexión en carretera, lo que dio origen al requerimiento no funcional **RNF-02 (disponibilidad sin conectividad)**: el sistema debe seguir alertando al conductor y no debe perder eventos si la red no está disponible, sincronizándolos al reconectarse. El modelado original asumía una

conexión permanente entre el cliente de captura y el API REST, lo cual contradecía este hallazgo y obligó a ajustar tres diagramas.

7.3. Tabla de validación

Tabla 6: Validación del modelado previo frente a los hallazgos de la elicitación de requerimientos.

Elemento revisado	**Concepción inicial**	**Hallazgo de la elicitación**	**Cambio realizado o justificación**
Conectividad del cliente de captura	El diagrama de componentes, el de despliegue y la secuencia de detección asumían envío directo y permanente de cada evento al API REST	La lluvia de ideas identificó el escenario de pérdida de conexión en carretera, lo que derivó en RNF-02	Se agregó el componente *Cola local de eventos* (buffer) en los diagramas de componentes y despliegue, y la rama *[Riesgo detectado · sin conexión]* en la secuencia de detección (Figura 3)
Actores del sistema	Conductor, Supervisor y Administrador en el diagrama de casos de uso	El análisis de interesados (sección 2) confirmó exactamente esos tres roles como usuarios directos; los demás interesados (empresa de transporte, docente, entidad reguladora) son externos y no interactúan directamente con el sistema	Sin cambios
Almacenamiento de datos del conductor	La clase `Evento` ya guardaba solo metadatos (tipo, severidad, fecha, score), sin video	La revisión de la Ley 1581 de 2012 (RNF-03) confirmó que no debe almacenarse video por tratarse de datos biométricos/personales	Sin cambios; el diseño ya cumplía el requerimiento antes de formalizarlo
Prioridad de los indicadores agregados	`Visualizar indicadores agregados` ya existía como caso de uso y como método de `Supervisor`, pero se consideraba una funcionalidad secundaria	La elicitación con el Supervisor de flota elevó RF-11 a prioridad Alta, por ser su necesidad principal para la toma de decisiones operativas	Sin cambio en el diagrama; se reclasifica su prioridad en el backlog para ubicarla en un sprint más temprano

Tabla 6: Validación del modelado previo frente a los hallazgos de la elicitación de requerimientos.

Elemento revisado	**Concepción inicial**	**Hallazgo de la elicitación**	**Cambio realizado o justificación**
Gestión de conductores y vehículos	CRUD ya modelado en `Administrador` (`gestionarConductores()`, `gestionarVehiculos()`) y en `Vehiculo.asignarConductor()`	Confirmado por la lluvia de ideas y por el rol de Administrador del sistema identificado en el análisis de interesados	Sin cambios

7.4. Diagramas actualizados

A partir de los hallazgos anteriores se actualizaron tres diagramas del modelado previo para representar el requerimiento RNF-02. Los demás diagramas (casos de uso, clases, secuencias de autenticación y de consulta de incidentes, actividades) no requirieron cambios y se mantienen como en el avance anterior.

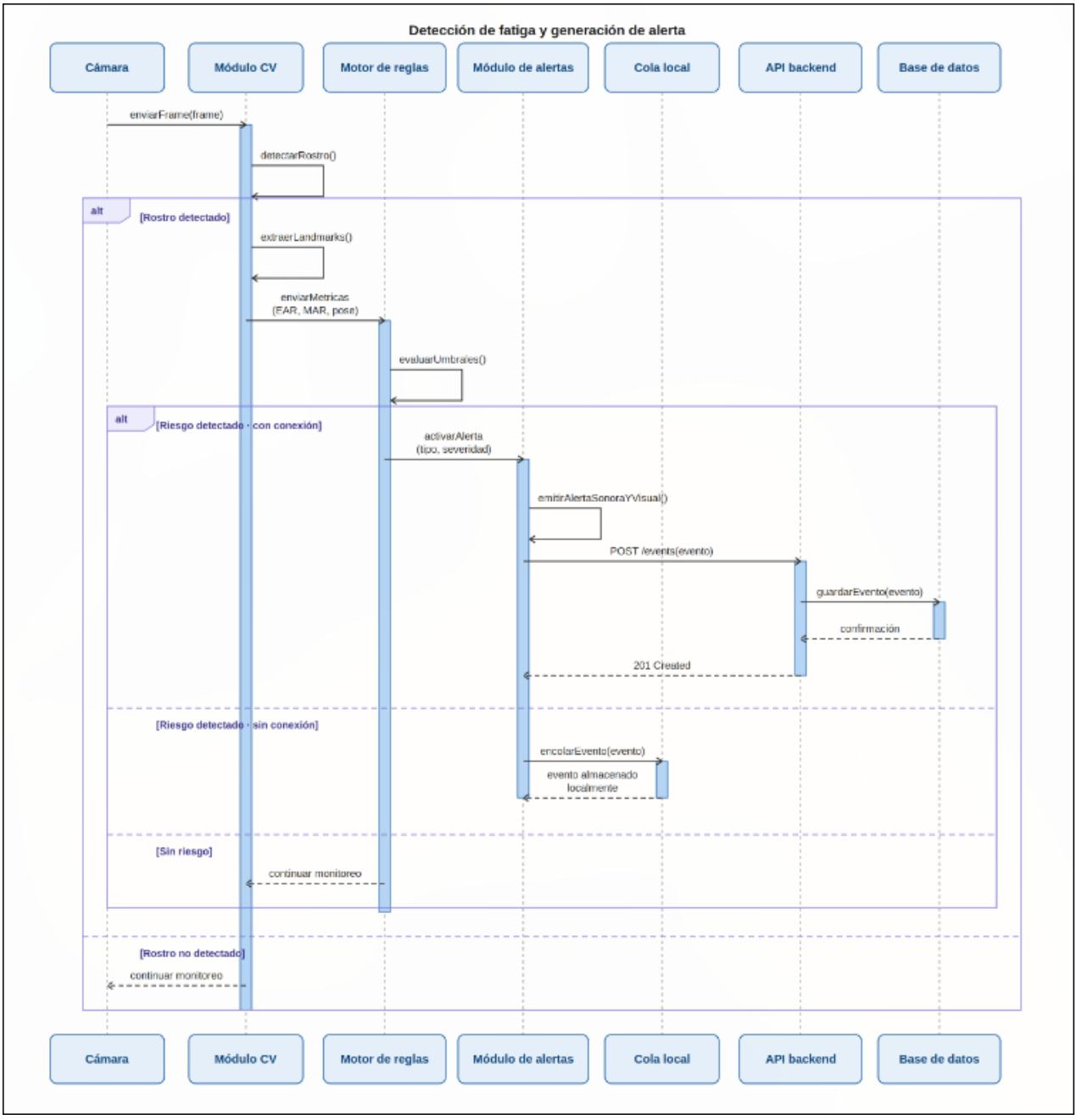


Figura 1: Diagrama de secuencia actualizado: detección de fatiga y generación de alerta. Se incorporó la rama *[Riesgo detectado · sin conexión]*, que encola el evento en el componente Cola local en vez de descartarlo.

Nota. Elaboración propia.

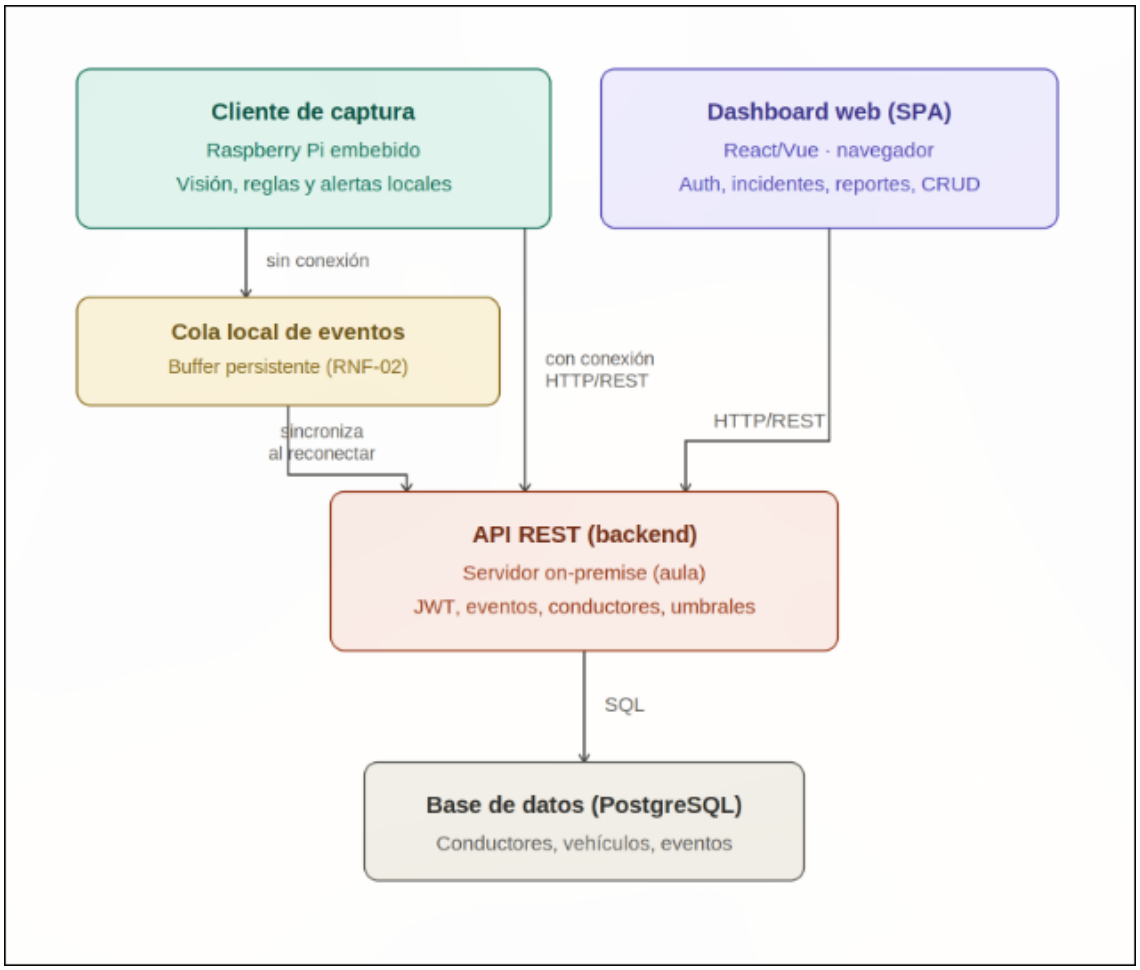


Figura 2: Diagrama de componentes actualizado. Se incorporó el componente Cola local de eventos (buffer) para cumplir RNF-02, con sincronización hacia el API REST al recuperar la conexión.

Nota. Elaboración propia.

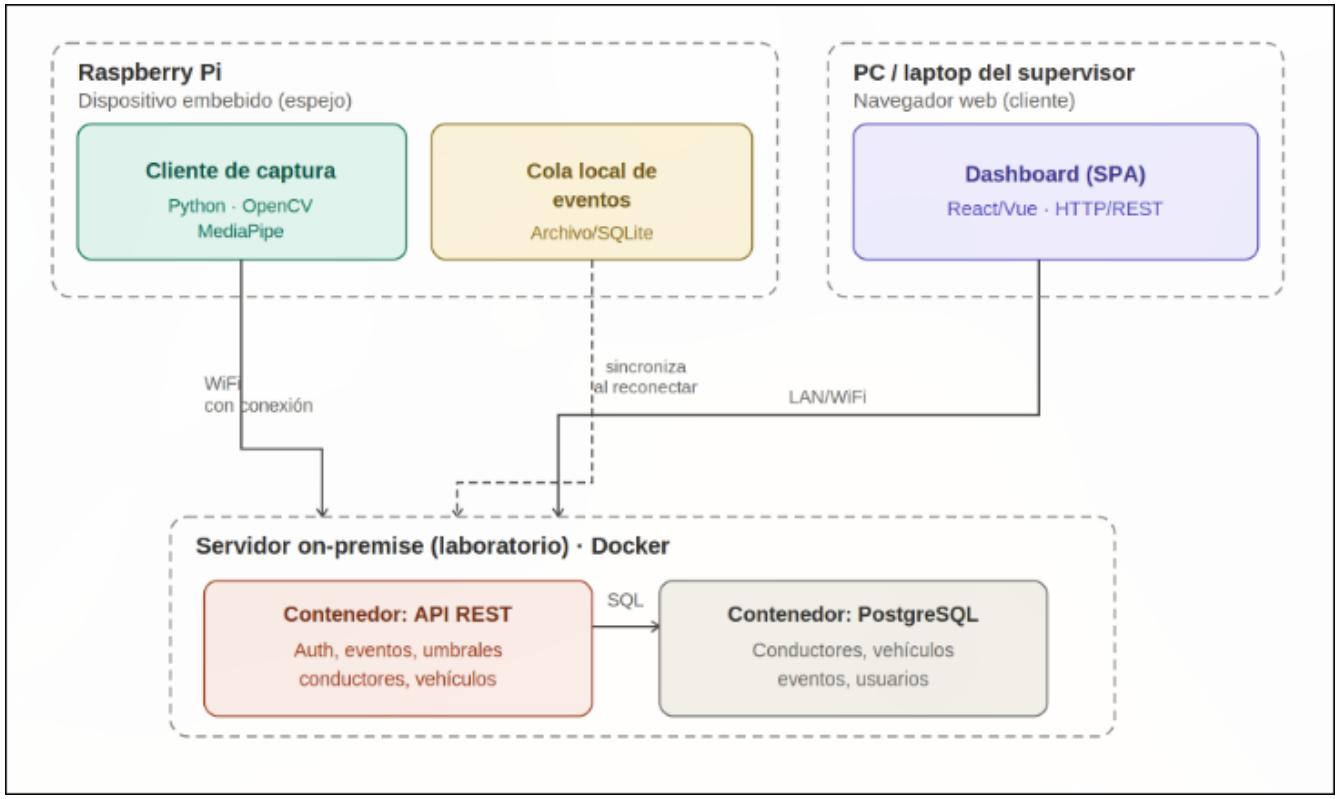


Figura 3: Diagrama de despliegue actualizado. Se representó el artefacto de almacenamiento local (Cola local de eventos) desplegado en la Raspberry Pi, junto con el flujo de sincronización hacia el servidor *on-premise*.

Nota. Elaboración propia.

8. Gestión del código fuente y del proyecto

8.1. Repositorio

El código fuente del proyecto se gestiona en GitHub, bajo la organización del curso:

`https://github.com/ISCOUTB/IS_DriveGuard`

El repositorio está organizado en los módulos `backend/`, `cv-module/`, `frontend/` y `docs/`, junto con `docker-compose.yml` para el despliegue por contenedores y un `README.md` con la descripción general del proyecto.

8.2. Herramienta de gestión del proyecto

Se utiliza **GitHub Projects**, el tablero Kanban integrado en el propio repositorio, con las columnas **Tareas** → **En Progreso** → **En Revisión** → **Finalizadas**. Cada tarea del tablero corresponde a un *issue* de GitHub vinculado a uno o más requerimientos (RF/RNF) del backlog.

8.3. Metodología ágil seleccionada y sustentación

El equipo adoptó **Scrum combinado con Kanban**:

- **Scrum.** El backlog ya viene priorizado con el método MoSCoW (sección 6), y el semestre académico impone un límite fijo de semanas, lo que encaja naturalmente con iteraciones cortas y entregables demostrables. Además, el docente revisa los avances durante las sesiones de clase, lo que funciona como un punto natural de *sprint review*.
- **Kanban.** Con un equipo de solo tres integrantes, imponer el ceremonial completo de Scrum (*planning*, *daily*, retrospectiva) representa una sobrecarga innecesaria. Un tablero de flujo continuo, visible dentro del mismo repositorio, es suficiente para coordinar el trabajo y se integra directamente con los *commits* y *pull requests*.

8.4. Aplicación práctica

Backlog. Los 19 requerimientos identificados (RF-01 a RF-11, RNF-01 a RNF-08), priorizados con MoSCoW, constituyen el backlog del proyecto. Cada uno se traduce en uno o más *issues* de GitHub, etiquetados según su prioridad (Must / Should / Could).

Roles. Cada integrante lidera un frente de trabajo diferenciado, con apoyo cruzado cuando se requiere:

- **Sebastián Ruiz** — entrenamiento del modelo y visión por computador (cliente de captura: EAR, MAR, pose de cabeza, score de fatiga).
- **Juan Diego Serpa** — backend (API REST, autenticación, base de datos).
- **Daniel Sánchez** — frontend (dashboard web).

Iteraciones. Se plantean seis sprints de dos semanas, alineados con las 12 semanas del curso:

Tabla 7: Plan de sprints propuesto para la primera versión funcional del sistema.

Sprint	**Semanas**	**Alcance**
1	1–2	Elicitación de requerimientos, modelado UML, configuración de repositorio y tablero (entregado en este avance)
2	3–4	RF-01, RF-02 — captura de video y detección facial/extracción de landmarks
3	5–6	RF-03 a RF-06 — cálculo de EAR, MAR, orientación de cabeza y score de fatiga
4	7–8	RF-07, RF-08, RF-09, RNF-02 — alerta local, registro de eventos, envío al backend y cola de sincronización sin conexión
5	9–10	RNF-03, RNF-05 — autenticación JWT y seguridad; RF-10 — CRUD de conductores y vehículos
6	11–12	RF-11 — indicadores agregados del dashboard; RNF-01 — ajuste de rendimiento; pruebas de aceptación

Reuniones de seguimiento. El equipo realiza una revisión semanal informal del tablero de GitHub Projects, complementada con la revisión de avance presentada en cada sesión de clase, la cual cumple la función de *sprint review* frente al docente.